

2nd-order approximation

$$f(\boldsymbol{w}) \approx f(\boldsymbol{w}_0) + \nabla f^T(\boldsymbol{w}_0)(\boldsymbol{w} - \boldsymbol{w}_0) + \frac{1}{2}(\boldsymbol{w} - \boldsymbol{w}_0)^T \overset{\text{Hessian}}{\nabla^2 f(\boldsymbol{w}_0)}(\boldsymbol{w} - \boldsymbol{w}_0)$$

$$\begin{bmatrix} 4 & -2 & -1 \\ -2 & 6 & -3 \\ -1 & -3 & 5 \end{bmatrix} \succ 0$$

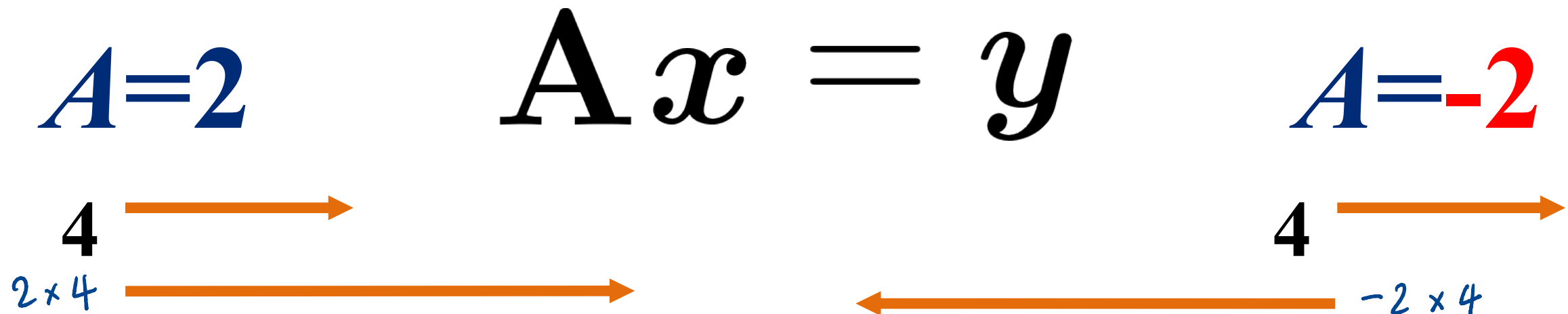
$$\begin{bmatrix} 4 > 0? & -2 > 0? & -1 > 0? \\ -2 > 0? & 6 > 0? & -3 > 0? \\ -1 > 0? & -3 > 0? & 5 > 0? \end{bmatrix} \succ 0$$

Mathematics Refresh: Multivariable Calculus and Linear Algebra

Positive numbers, when you think of them as **operators**, are really the ones that keep their output *on the same side* as their input

In 1D, saying that x and y are on the same side $\Leftrightarrow xy > 0$. But $y = Ax$, so

In 1D, saying that x and y are on the same side $\Leftrightarrow xy > 0$. But $y = Ax$, so
 $xy = x(Ax) = Ax^2$ and $Ax^2 > 0, \forall x \neq 0 \Leftrightarrow A > 0$

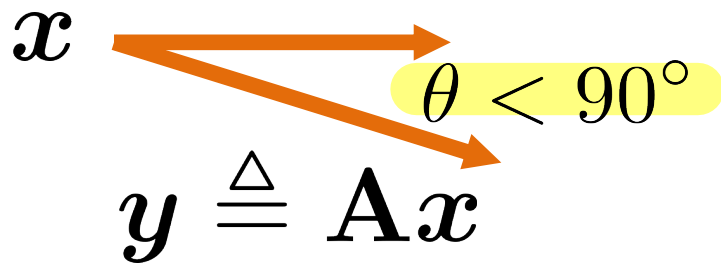


$\mathbf{A} \succ 0$ if and only if $\forall \mathbf{x} \neq \mathbf{0}$, $\mathbf{y} \triangleq \mathbf{A}\mathbf{x}$ stays on the same side as $\mathbf{x}$
(Positive Definite)

the vector $\mathbf{y}$ that $\mathbf{A}$ outputs

$$\mathbf{x}^T (\mathbf{A}\mathbf{x}) > 0$$

$$\mathbf{x}^T \mathbf{A} \mathbf{x} > 0$$



$\mathbf{A} \succeq 0$ if and only if $\mathbf{x}^T \mathbf{A} \mathbf{x} \geq 0, \forall \mathbf{x} \neq \mathbf{0}$
(Positive Semidefinite)

Constrained vs. Unconstrained Optimization

Unconstrained Optimization

$$\min_{\mathbf{w} \in \mathbb{R}^n} f(\mathbf{w})$$

A clearly defined objective function,
 $f : \mathbb{R}^n \rightarrow \mathbb{R}$

Decision Variables:

$$\mathbf{w} \in \mathbb{R}^n$$

input vectors n entries

eg. ML
→ params can be
+ve / -ve

Constrained Optimization

$$\min_{\mathbf{w} \in \mathbb{R}^n} f(\mathbf{w})$$

subject to

$$\begin{cases} c_i(\mathbf{w}) = 0, & \text{for } i \in \mathcal{E}, \\ c_i(\mathbf{w}) \geq 0, & \text{for } i \in \mathcal{I}. \end{cases}$$

Constraints define the feasible region

$$f \in C^1$$

at minimum for this course,
and sometimes we require $f' \in C^2$

C^1 : The first-order derivative is a continuous function

C^2 : The second-order derivative is a continuous function

eg. Portfolio Management (there is initial budget)

Machine Learning Training as an Unconstrained Optimization Problem

Training Dataset (^{training examples} m rows
and $d+1$ columns)
^{features} $x_1 \ x_2 \ x_3 \ \dots \ x_d$ y ^{target variable}



$m \times d$

Model: $g(\boldsymbol{w}|\boldsymbol{x})$

where $\boldsymbol{w}$ is the parameter vector
that need to be determined and
 $\boldsymbol{x} = (x_1, x_2, \dots, x_d)$

$$\min_{\boldsymbol{w} \in \mathbb{R}^n} f(\boldsymbol{w}) = \frac{1}{m} \sum_{i=1}^m \left[y^{(i)} - g(\boldsymbol{w}|\boldsymbol{x}^{(i)}) \right]^2$$

Machine learning eventually reduce
to solving optimization problems

First-Order Necessary Condition

$\min_{w \in \mathbb{R}^n} f(w)$ can be a hard problem in general.

are 1B points enough?

GPU brute force? Sampling 1B points?

e.g., $\min_{w \in (-\infty, +\infty)} f(w) = w^4/50 - w^2 - \sin(8w + 5)e^{\cos w}$

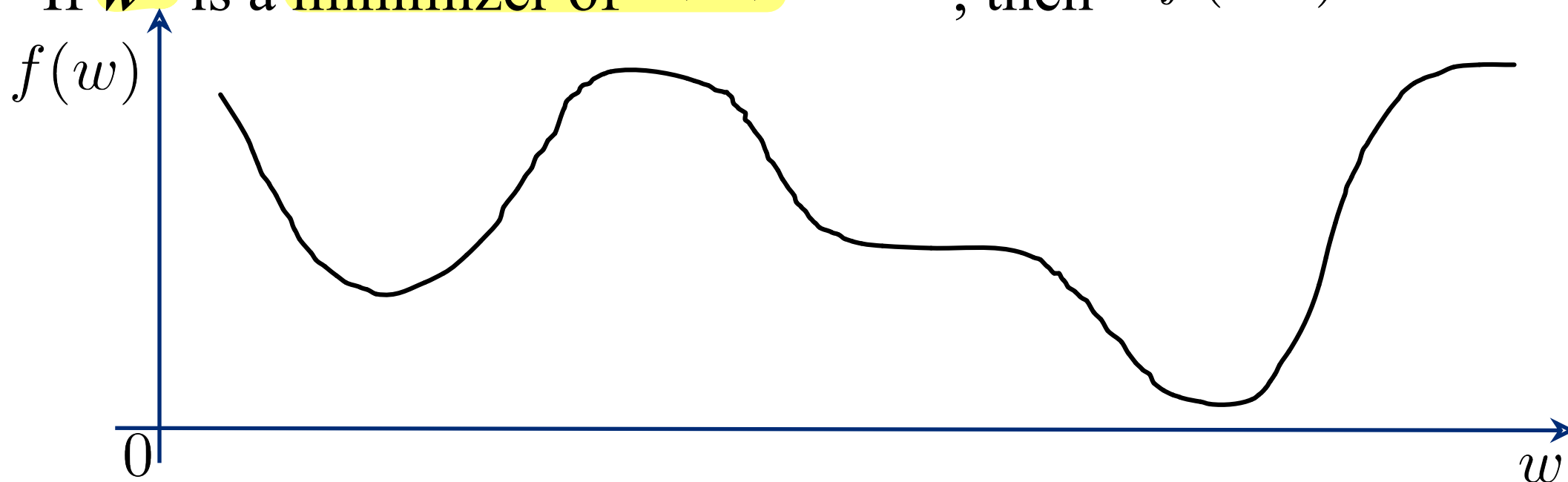
not scalable to
n dimensional problems

Range of the points?

1B grid pts in 10D $\approx$ 8 values per coordinate

Theorem (First-Order Necessary Condition)

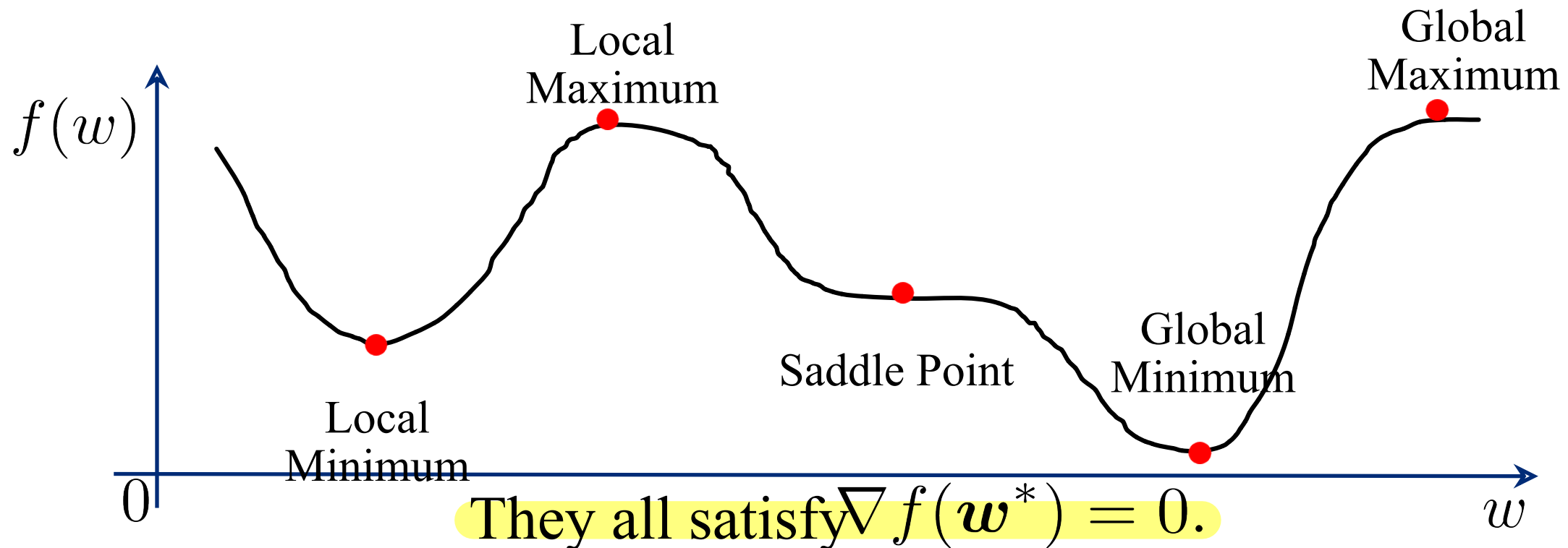
If w^* is a minimizer of $f(w) \in C^1$, then $\nabla f(w^*) = 0$.



Global vs. Local Minimum

Local Minimum: A point w^* is a local minimizer if there exists a neighborhood $\mathcal{N}(w^*)$ such that $f(w^*) \leq f(w)$, $\forall w \in \mathcal{N}(w^*)$.

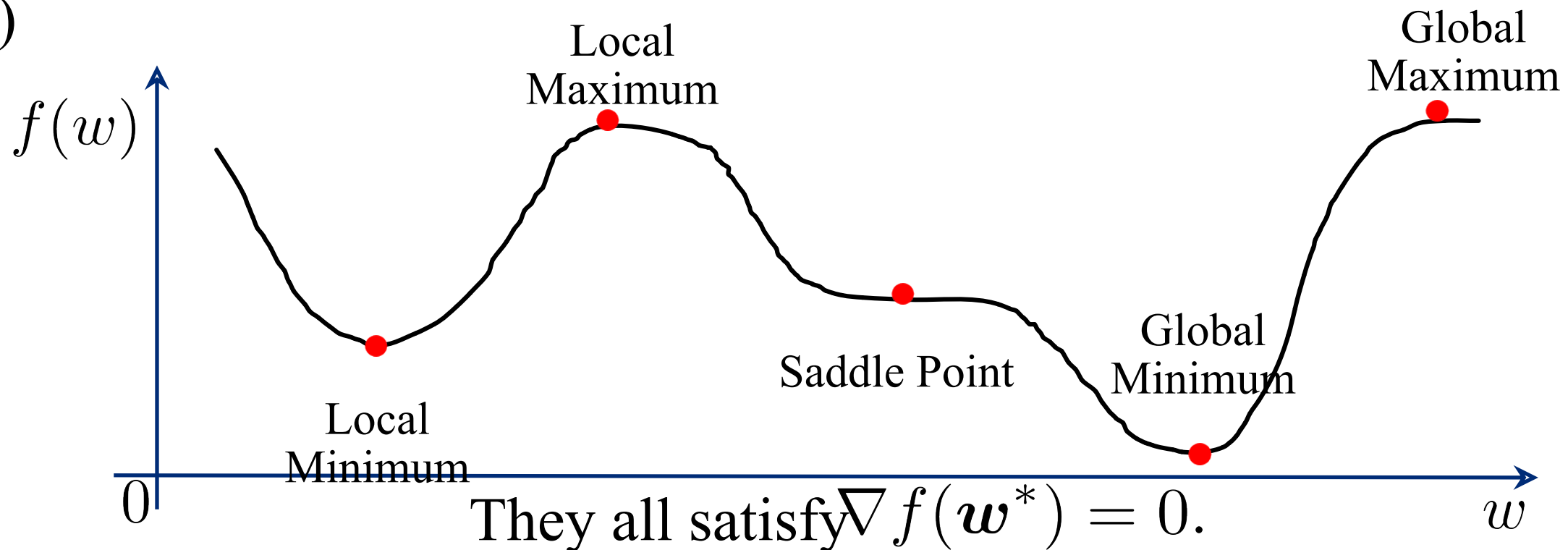
Global Minimum: A point w^* is a global minimizer if $f(w^*) \leq f(w)$, $\forall w$.



What Can an Optimization Algorithm Guarantee?

Goal: Design a method (an algorithm) to find $\hat{w}$, *s.t.* $\|\nabla f(\hat{w})\| \leq \epsilon$.
candidates for minimizer

Ruling out local maxima and saddle points requires more information about the objective function (e.g., Hessian, 2nd order partial derivative test), which can be costly to compute. Global minimum? Forget about it!! (unless you know more info about f)



Unconstrained Optimization with Exact Solution: e.g., **Linear Model**

A case where we can solve $\nabla f(\mathbf{w}) = 0$ exactly

$$\min_{\mathbf{w} \in \mathbb{R}^n} f(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \left[y^{(i)} - g(\mathbf{w} | \mathbf{x}^{(i)}) \right]^2$$

$$g(\mathbf{w} | \mathbf{x}) = \mathbf{w}^T \mathbf{x} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d$$

$\mathbf{w} = (w_0, w_1)$ *decision variables to optimise over* $g(\mathbf{w} | \mathbf{x}) = w_0 + w_1 x_1$

$$\begin{aligned} f(\mathbf{w}) &= \frac{1}{m} \sum_{i=1}^m \left[y^{(i)} - (w_0 + w_1 x_1^{(i)}) \right]^2 \\ &= \frac{1}{m} \left([y^{(1)} - (w_0 + w_1 x_1^{(1)})]^2 + [y^{(2)} - (w_0 + w_1 x_1^{(2)})]^2 + \cdots + [y^{(m)} - (w_0 + w_1 x_1^{(m)})]^2 \right) \end{aligned}$$

This is **quadratic in w_0 and w_1**

Unconstrained Optimization with Exact Solution: e.g., Linear Model

A case where we can solve $\nabla f(\mathbf{w}) = 0$ exactly

$$\begin{aligned} f(\mathbf{w}) &= \frac{1}{m} \sum_{i=1}^m \left[y^{(i)} - (w_0 + w_1 x_1^{(i)}) \right]^2 \\ &= \frac{1}{m} \left([y^{(1)} - (w_0 + w_1 x_1^{(1)})]^2 + [y^{(2)} - (w_0 + w_1 x_1^{(2)})]^2 + \cdots + [y^{(m)} - (w_0 + w_1 x_1^{(m)})]^2 \right) \end{aligned}$$

$$\nabla f(\mathbf{w}) = \left(\frac{\partial f(\mathbf{w})}{\partial w_0}, \frac{\partial f(\mathbf{w})}{\partial w_1} \right)$$

$$\frac{\partial f(\mathbf{w})}{\partial w_0} = \frac{-2}{m} \left(y^{(1)} - (w_0 + w_1 x_1^{(1)}) + y^{(2)} - (w_0 + w_1 x_1^{(2)}) + \cdots + y^{(m)} - (w_0 + w_1 x_1^{(m)}) \right) = 0$$

$$\frac{\partial f(\mathbf{w})}{\partial w_1} = \frac{-2}{m} \left(\left[y^{(1)} - (w_0 + w_1 x_1^{(1)}) \right] x_1^{(1)} + \left[y^{(2)} - (w_0 + w_1 x_1^{(2)}) \right] x_1^{(2)} + \cdots + \left[y^{(m)} - (w_0 + w_1 x_1^{(m)}) \right] x_1^{(m)} \right) = 0$$

$$w_1^* = \frac{\sum_i^m (x^{(i)} - \bar{x})(y^{(i)} - \bar{y})}{\sum_i^m (x^{(i)} - \bar{x})^2} \quad w_0^* = \bar{y} - w_1^* \bar{x} \quad \text{where} \quad \bar{x} = \frac{1}{m} \sum_{i=1}^m x^{(i)} \quad \bar{y} = \frac{1}{m} \sum_{i=1}^m y^{(i)}$$

Unconstrained Optimization with Exact Solution: e.g., Linear Model

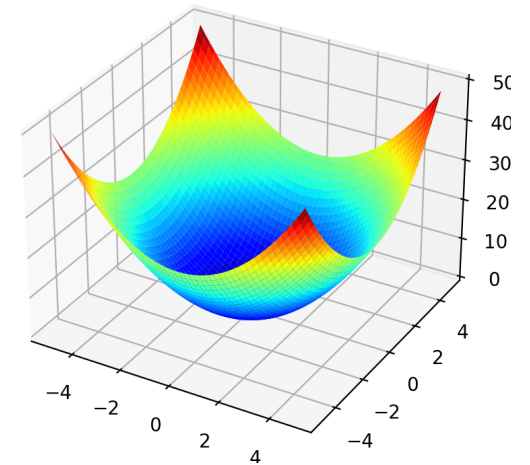
$$\min_{\mathbf{w}} f(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)})^2 = \frac{1}{m} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = \frac{1}{m} (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

training features

$$\mathbf{X} = \begin{bmatrix} -(\mathbf{x}^{(1)})^T - \\ -(\mathbf{x}^{(2)})^T - \\ \vdots \\ -(\mathbf{x}^{(m)})^T - \end{bmatrix} \quad \text{where } \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

target vals

each row = one instance



$$\nabla f(\mathbf{w}^*) = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\mathbf{w}^* - \mathbf{y}) = 0 \Leftrightarrow \mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad \text{provided that } \mathbf{X}^T \mathbf{X} \text{ is invertible}$$

$f(\mathbf{w})$ is a convex function. Hence, the local minimum point is also a global minimum.

$f(\mathbf{w})$ is a quadratic function. Hence, the local (global) minimizer has a closed-form expression

Unconstrained Optimization without a Closed-form Solution: Logistic Model

Training Dataset (m rows
and $d+1$ columns)

x_1 x_2 x_3 $\dots$ x_d y



$m \times d$

Model: $g(\mathbf{w}|\mathbf{x})$

where $\mathbf{w}$ is the parameter vector
that need to be determined and
 $\mathbf{x} = (x_1, x_2, \dots, x_d)$

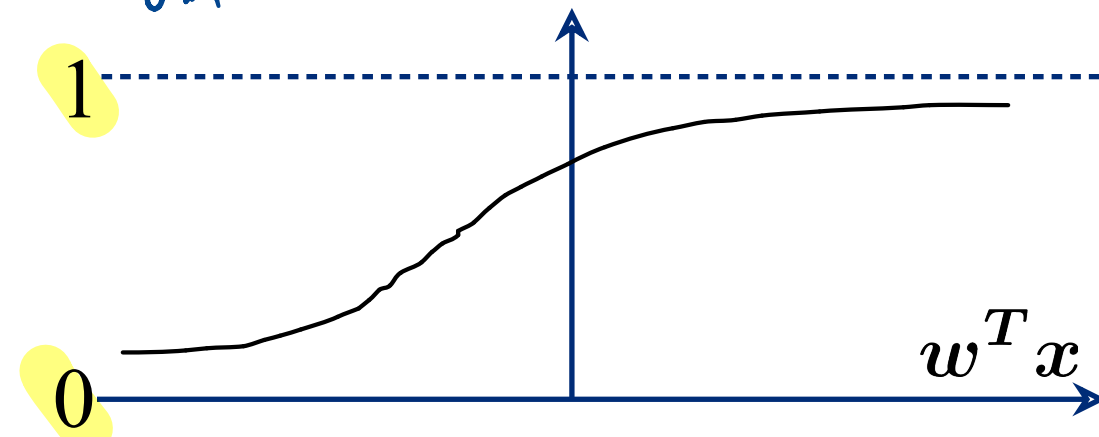
$$g(\mathbf{w}|\mathbf{x}) = \frac{e^{w^T \mathbf{x}}}{1 + e^{w^T \mathbf{x}}}$$

> 0
> numerator

linear model in w

$$w^T \mathbf{x} = w_1 x_1 + w_2 x_2 + \dots + w_d x_d$$

always between 0 & 1



Unconstrained Optimization without a Closed-form Solution: Logistic Model

$$\mathbb{P}\{Y^{(i)} = 1 | \mathbf{x} = \mathbf{x}^{(i)}\} = g(\mathbf{w} | \mathbf{x}^{(i)}) = \frac{e^{\mathbf{w}^T \mathbf{x}^{(i)}}}{1 + e^{\mathbf{w}^T \mathbf{x}^{(i)}}} = p_i$$

$$\mathbb{P}\{Y^{(i)} = 0 | \mathbf{x} = \mathbf{x}^{(i)}\} = 1 - g(\mathbf{w} | \mathbf{x}^{(i)}) = 1 - \frac{e^{\mathbf{w}^T \mathbf{x}^{(i)}}}{1 + e^{\mathbf{w}^T \mathbf{x}^{(i)}}} = 1 - p_i$$

Likelihood of observing instance i in the training dataset, given the input feature $\mathbf{x}^{(i)}$

$$\mathbb{P}\{Y^{(i)} = y | \mathbf{x} = \mathbf{x}^{(i)}\} = p_i^y (1 - p_i)^{1-y} = \left(\frac{e^{\mathbf{w}^T \mathbf{x}^{(i)}}}{1 + e^{\mathbf{w}^T \mathbf{x}^{(i)}}} \right)^y \left(\frac{1}{1 + e^{\mathbf{w}^T \mathbf{x}^{(i)}}} \right)^{1-y}$$

But here we have a total of m instances in our training dataset

Unconstrained Optimization without a Closed-form Solution: Logistic Model

We view the training data points as independent draws from the same population

$$\begin{aligned}\mathcal{L}(\mathbf{w}) &= \prod_{i=1}^m \mathbb{P}\{Y^{(i)} = y^{(i)} | \mathbf{x} = \mathbf{x}^{(i)}\} = \prod_{i=1}^m p_i^{y^{(i)}} (1 - p_i)^{1-y^{(i)}} \\ &= \prod_{i=1}^m \left(\frac{e^{\mathbf{w}^T \mathbf{x}^{(i)}}}{1 + e^{\mathbf{w}^T \mathbf{x}^{(i)}}} \right)^{y^{(i)}} \left(\frac{1}{1 + e^{\mathbf{w}^T \mathbf{x}^{(i)}}} \right)^{1-y^{(i)}}\end{aligned}$$

We choose $\mathbf{w}$ to maximize this joint probability: Maximum Likelihood Estimation (MLE)

$$\max_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \prod_{i=1}^m \left(\frac{e^{\mathbf{w}^T \mathbf{x}^{(i)}}}{1 + e^{\mathbf{w}^T \mathbf{x}^{(i)}}} \right)^{y^{(i)}} \left(\frac{1}{1 + e^{\mathbf{w}^T \mathbf{x}^{(i)}}} \right)^{1-y^{(i)}}$$

A technical problem: when m is large, then $\mathcal{L}(\mathbf{w})$ is very close to 0, which can cause numerical underflow issue.

computer cannot distinguish bwn the very small values

Unconstrained Optimization without a Closed-form Solution: Logistic Model

We take the natural logarithm of the objective function

$$\max_w \mathcal{L}(w) = \prod_{i=1}^m p_i^{y^{(i)}} (1 - p_i)^{1-y^{(i)}} \longleftrightarrow \max_w \ln \left(\prod_{i=1}^m p_i^{y^{(i)}} (1 - p_i)^{1-y^{(i)}} \right)$$

(decr.)
original fn increasing,
log transform still incr.
(decr.)

Since $\ln(\cdot)$ is a monotone transformation, the original problem and the new problem are equivalent.

vs product of fractions $\rightarrow 0$
avoids problem of overfit collapsing towards 0
more terms, further from 0

$$\max_w \sum_{i=1}^m \left[y^{(i)} \ln p_i + (1 - y^{(i)}) \ln(1 - p_i) \right]$$



There is no simple formula for w^* anymore, we have to develop an algorithm to solve this optimization problem iteratively

improve iteratively until stopping criteria met

$$\text{where } p_i = \frac{e^{w^T \mathbf{x}^{(i)}}}{1 + e^{w^T \mathbf{x}^{(i)}}}$$

$$\min_w f(w) = \frac{1}{m} \sum_{i=1}^m \left[y^{(i)} - w^T \mathbf{x}^{(i)} \right]^2 \text{ has a closed-form solution}$$

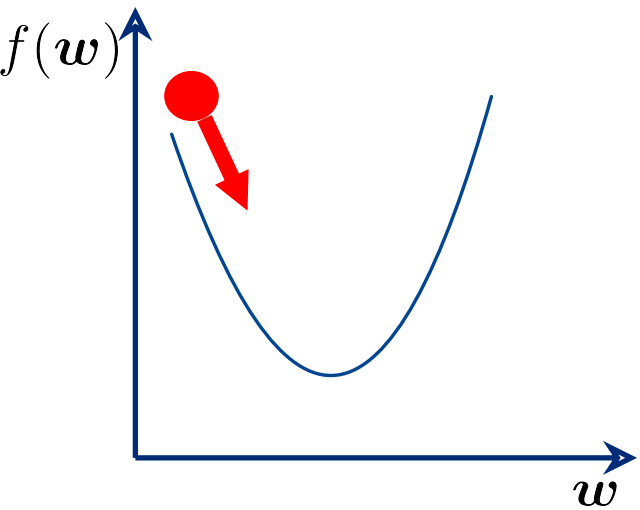
What Happens Inside .fit()? Logistic Regression in scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
```

```
X, y = load_iris(return_X_y=True)
mask = y < 2
X, y = X[mask], y[mask]
```

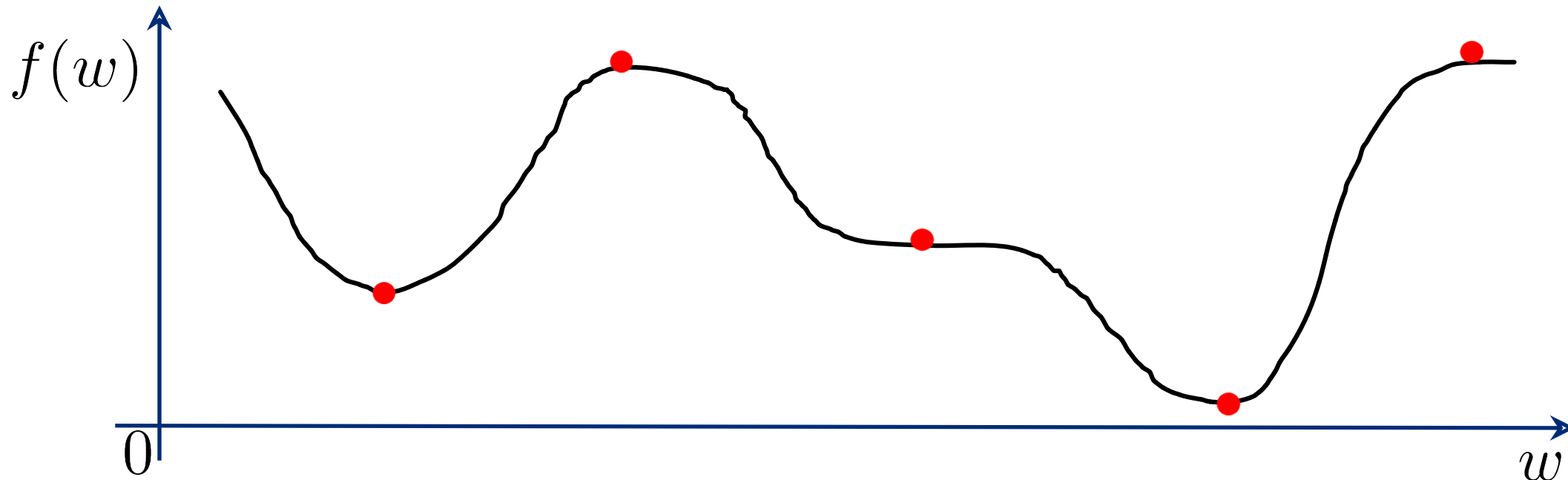
```
clf = LogisticRegression(
    penalty="l2",
    solver="lbfgs",
    max_iter=100
).fit(X, y)
```

Gradient Descent: Descent Direction

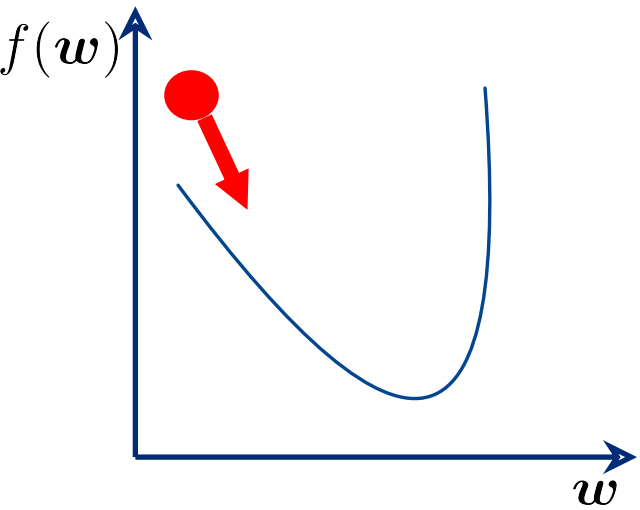


Definition (Descent Direction): For a given point $w \in \mathbb{R}^n$, a direction $d \in \mathbb{R}^n$ is called a *descent direction*, if there exists $\alpha > 0$ such that $f(w + \alpha d) < f(w)$, $\forall \alpha \in (0, \bar{\alpha})$.

Interpretation: Every sufficiently small positive step along d decreases the objective function.



Gradient Descent: Which Direction Is Steepest?



In non-linear optimization:

You don't get to see the surface,
i.e., the full picture of $f(w)$.

You only have local information
at the point of evaluation.

General method for non-linear optimization

Start at $w(0)$

Take a step along steepest slope

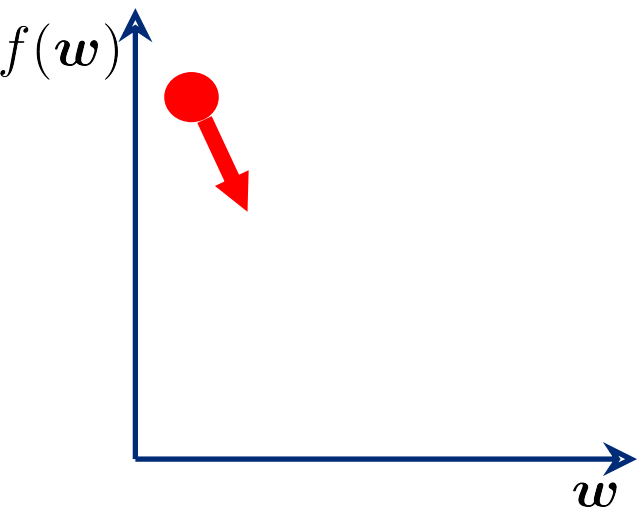
Fixed step size

$$w(1) = w(0) + \eta \cdot \hat{v}$$

What is the direction $\hat{v}$?

how
Far? where?

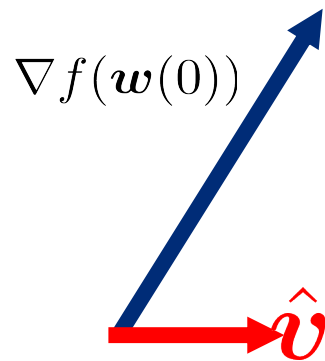
Gradient Descent: Why the Negative Gradient?



In non-linear optimization:

You don't get to see the surface,
i.e., the full picture of $f(w)$

You only have local information
at the point of evaluation



Formula for the direction $\hat{\mathbf{v}}$

Recall : $f(\mathbf{w}) \approx f(\mathbf{w}_0) + \nabla f^T(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0)$

$$\Delta f(\mathbf{w}) = f(\underbrace{\mathbf{w}(0) + \eta \hat{\mathbf{v}}}_{\mathbf{w}(1)}) - f(\mathbf{w}(0))$$

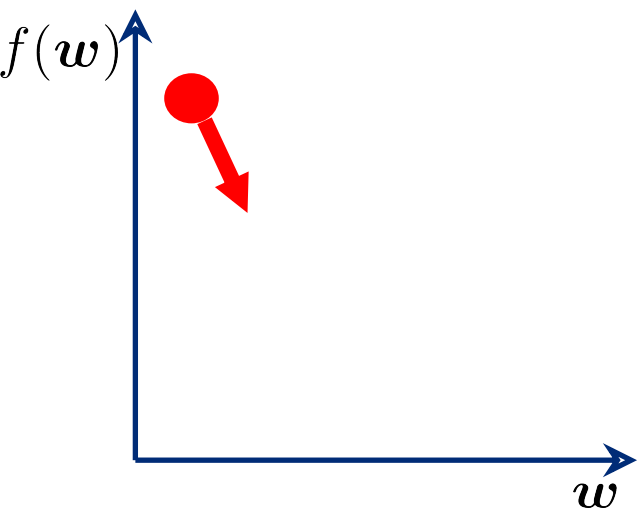
$$= \underbrace{\eta \nabla f^T(\mathbf{w}(0)) \hat{\mathbf{v}}}_{\text{first order Taylor expansion}} + \underbrace{O(\eta^2)}_{\text{neglect this}}$$

Directional derivative of f at point $\mathbf{w}(0)$ and along the direction $\hat{\mathbf{v}}$

$$\geq -\eta \|\nabla f(\mathbf{w}(0))\|, \quad \text{for any } \hat{\mathbf{v}}$$

Since $\hat{\mathbf{v}}$ is a unit vector $\hat{\mathbf{v}} = - \frac{\nabla f(\mathbf{w}(0))}{\|\nabla f(\mathbf{w}(0))\|}$
normalise

Gradient Descent: How Far Should We Move?

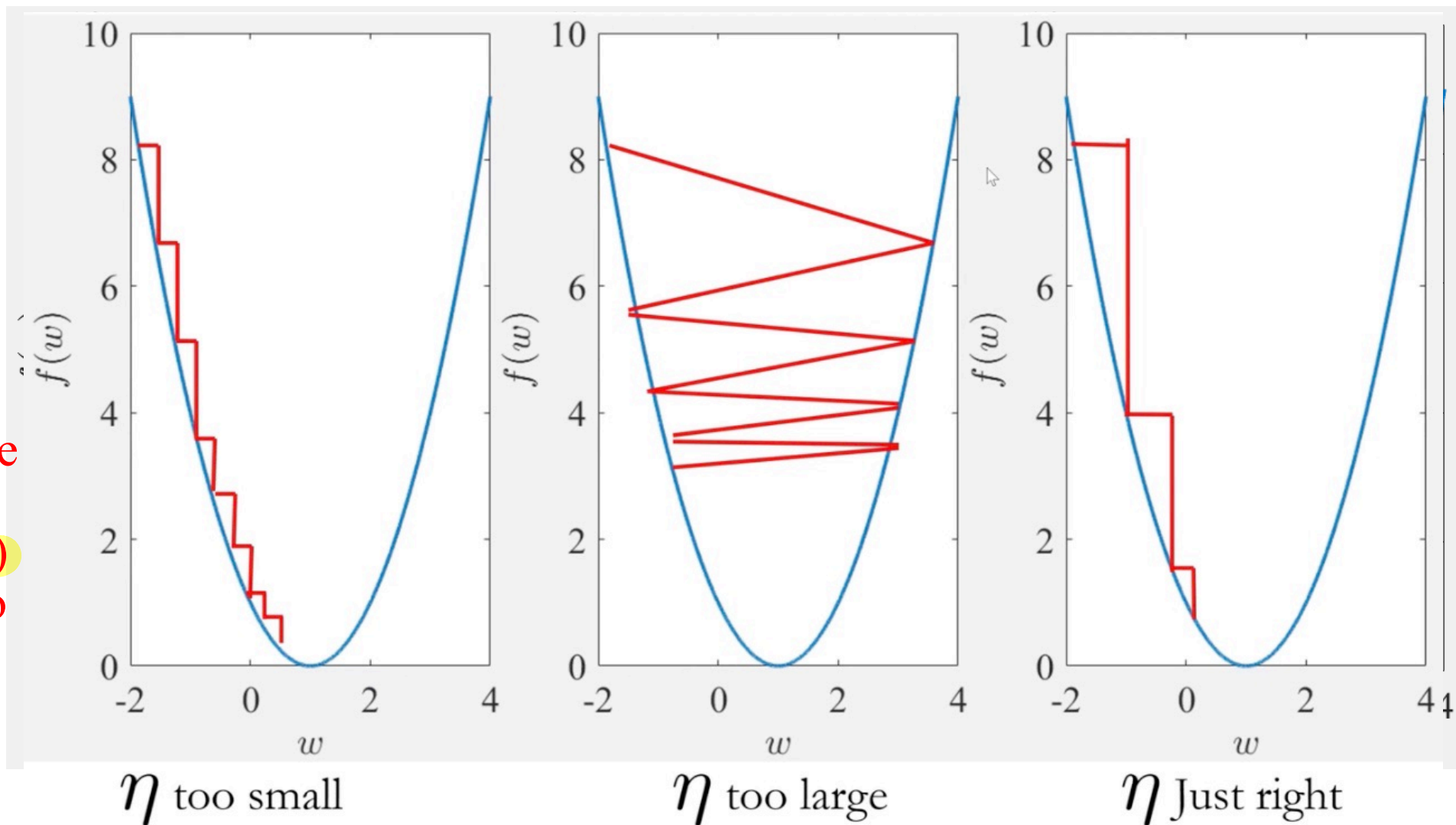


Fixed-size step?

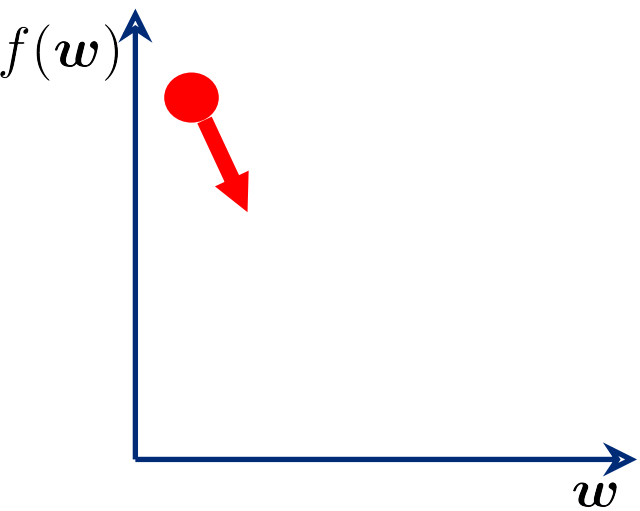
How η affects the algorithm:

In optimization, I charge you for: (a) the final value you arrived at; (b) how long it takes you to get there

η should increase with the $|\text{slope}|$



Gradient Descent: Steepest Descent



Easy Implementation

$$\text{Instead of } \mathbf{w}(1) - \mathbf{w}(0) = \Delta \mathbf{w} = \eta \hat{\mathbf{v}} \\ = -\eta \frac{\nabla f(\mathbf{w}(0))}{\|\nabla f(\mathbf{w}(0))\|}$$

$$\text{setting } \eta = \alpha \|\nabla f(\mathbf{w}(k))\|$$

Fixed Learning Rate α 0.1 works most of the time for convex
(step size)

$$\text{Then } \Delta \mathbf{w} = -\alpha \nabla f(\mathbf{w}(k))$$

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \alpha \nabla f(\mathbf{w}(k))$$

Gradient Descent: Steepest Descent in Logistic Regression

Step 1. Initialize $\mathbf{w}(0)$ (e.g., randomly pick a vector)

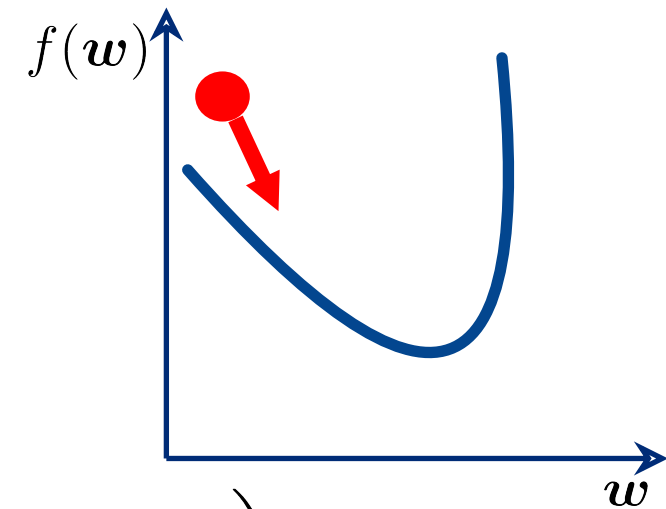
Step 2. For iteration $k = 0, 1, 2, \dots$

Compute the gradient

$$\nabla f(\mathbf{w}(k)) = -\frac{1}{m} \sum_{i=1}^m \mathbf{x}^{(i)} \left(\frac{1}{1 + \exp\{\mathbf{w}^T \mathbf{x}^{(i)}\}} + y^{(i)} - 1 \right)$$

Update: $\mathbf{w}(k+1) = \mathbf{w}(k) - \alpha \nabla f(\mathbf{w}(k))$

Termination: Stop when $\|\nabla f(\mathbf{w}(k))\|$ is small or k exceeds a cutoff.



Gradient Descent in General: Initialization and Local Minima

Initialize $w(0)$

Set a target minimum value for $f(w)$???

$$f(w(k)) \leq f_{target}$$

Termination criteria

In practice, combine a convergence test with a hard computational limit.

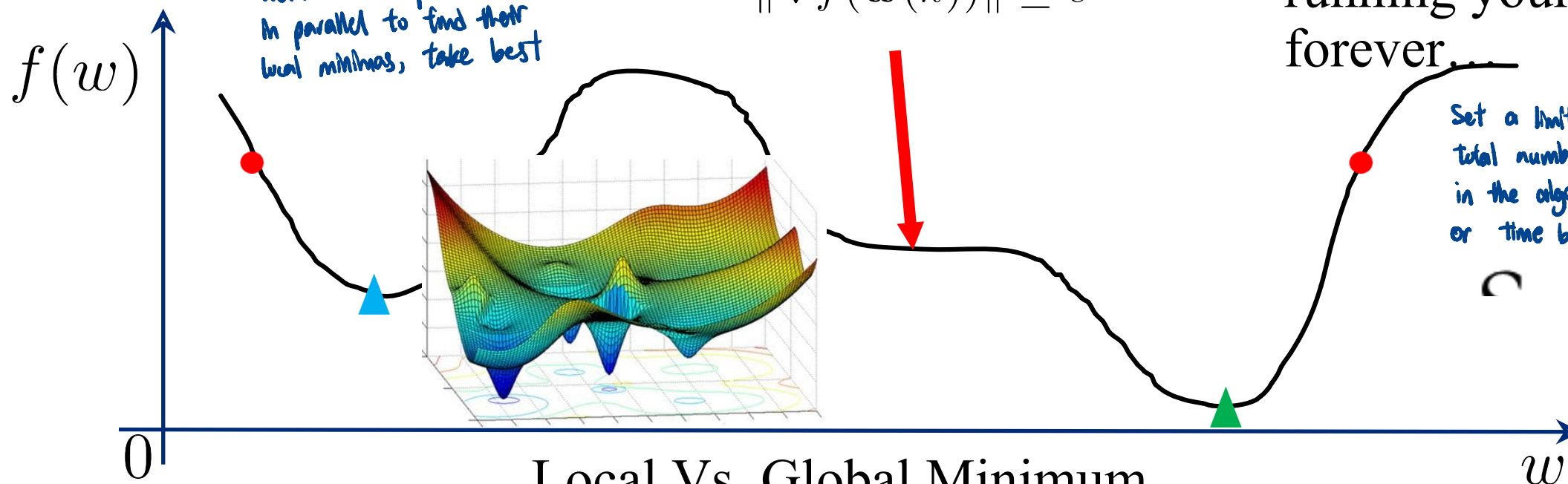
you can start algo from several points in parallel to find their local minimas, take best

May stop prematurely

$$\|\nabla f(w(k))\| \leq \epsilon$$

You may end up with running your algorithm forever...

Set a limit on the total number of steps in the algorithm $k \geq k_{max}$ or time budget exceeded



Local Vs. Global Minimum

Some Comments on Gradient Descent in General

First-order information Vs. Second-order information (of the objective function)

Gradient Descent can be slow for some problems since it only uses the gradient

The 2nd order information is not free

Stochastic Gradient Descent

Newton, Quasi-Newton: BFGS, L-BFGS etc.

But when you judge one method versus the other in optimization, don't be excited about the sophisticated method, because you will be charged for it

Fast Convergence of the Algorithm
(Fewer Iterations)

Vs. Higher Computational Cost per Iteration

The Ladder of Optimization Goals

In theory, we want to design algorithms to:

find **the global minimum** of the objective function (**NP-hard in general**, unless the function has some special nice properties, e.g., global convexity)

find **a local minimum** of the objective function (which still requires certain conditions on the objective function) *eg. Hessian matrix positive definite to exclude saddle point*

find **a stationary point** of the objective function, i.e. $\|\nabla f(\mathbf{w}(k))\| \leq \epsilon$
(this is what we typically do) *

In practice, we want to design algorithms to:

Return a sufficiently good point within a **limited computational budget**.
(**Whatever works before the deadline**)

Two Fundamental Frameworks for Choosing the Next Iterate

Line Search and Trust Region

Line Search:

Given a descent direction $\mathbf{p}(k)$

e.g., steepest descent $-\nabla f(\mathbf{w}(k))$

Pick a step size α such that

$$f(\mathbf{w}(k) + \alpha \mathbf{p}(k)) < f(\mathbf{w}(k))$$

Line search starts by fixing a descent direction $\mathbf{p}(k)$ and then identifying an appropriate distance, the step size α .

Trust Region:

$$\begin{aligned} \min_{\mathbf{p}} \quad & m_k(\mathbf{w}(k) + \mathbf{p}) \\ \text{s.t.} \quad & \|\mathbf{p}\| \leq \delta \end{aligned}$$

Trust region first chooses a max distance δ (the trust-region radius) and then seeks a direction and step that attains the best improvement possible subject to this distance constraint